Working with Claude Code in This Project

An onboarding guide for human researchers

Esteban Degetau
World Bank
estebandegetau@gmail.com

Agustín Samano
World Bank
asamanopenaloza@worldbank.org

June 28, 2026

Why this document exists

This project uses Claude Code extensively: for writing R functions, managing a reproducible pipeline, developing LLM codebooks, and maintaining documentation. Claude Code can do most of the tasks involved in empirical research. The harder question is what it *should* do versus what the human researcher must own.

This document answers that question for this project. It starts with principles, then maps those principles onto concrete systems we've built.

Part 1: Principles

The chain of human ownership

Four responsibilities belong to the human researcher. They form a chain: each one depends on the ones before it.

1. Meaning. The human both *receives* and *confers* meaning. Receiving: understanding why fiscal policy's impact on growth, employment, and investment in emerging economies matters. Conferring: deciding that these numbers in the dataset *are* exogenous fiscal shocks that causally identify fiscal policy. An LLM can pattern-match against codebook definitions, but it does not *know* what exogeneity means for causal identification. It does not understand that getting this wrong invalidates the entire downstream analysis. Knowledge does not exist until *someone* claims it and can be held accountable for the claim.

2. Learning. The human must learn enough to own the work. This is not a moral imperative but a practical one. If you defer all intellectual engagement to the end, you face a misaligned incentive to share output you do not understand. The project infrastructure should be a commitment device that forces intellectual engagement at the right moments. Not everywhere, but at every point that is central to the contribution.

A heuristic for deciding where to engage deeply: **how central is the method to the research contribution?** PDF extraction is instrumental; you can delegate it fully and verify outputs against your mental model of what well-extracted text

looks like. But codebook design is the intellectual contribution of this project. You must understand in detail how and why the codebooks work, and when they might not. Delegate the instrumental. Own the core.

3. Credibility. The depth of your involvement determines the depth at which you can credibly defend your work. When a referee asks "why did you use a detection framing instead of topic classification for C1?", you need to have *lived through* iteration 9′s catastrophic 29.1% recall, understood the dominant-topic heuristic diagnosis, and made the decision to reframe. You choose where to go deep and where to delegate, but you own the consequences of that choice.

4. Decision ownership. Claude can analyze, propose, challenge, and trace implications. But the commit ("we will do this, not that") belongs to the human. Decisions are auditable in ways that judgment is not. The iteration log records every decision with a date, context, and human rationale. Someone can examine whether the decision was well-reasoned. The person who made the decision faces the consequences: if removing the enacted filter causes problems downstream, that is your problem to solve. Claude proposed it, you approved it, your name is on the paper.

The core heuristic

AI owns tasks where errors are recoverable. Humans own tasks where errors compound silently.

Code has tests. Metrics can be recomputed. YAML can be re-validated. If Claude gets these wrong, you find out quickly and fix it cheaply.

A bad research design does not throw an error. A wrong interpretation gets baked into the next iteration's decision field and shapes all future work. A cost incurrence cannot be un-spent. The infrastructure we have built forces human involvement precisely at these high-consequence points.

What honest research with AI looks like

The heuristics we have built are mechanisms for honesty:

- The **iteration log** records failures, not just successes. Eleven iterations, eight of which failed.
- The **delta log** forces you to acknowledge when implementation contradicts your plan, rather than quietly drifting.
- **Strategy sync** makes you justify decisions under adversarial questioning, not just rationalize them after the fact.
- **One change at a time** prevents hiding behind a bundle of changes where you cannot tell what worked.
- The **pre-flight checklist** prevents running experiments on uncommitted code where you cannot reproduce the result.

These are not efficiency tools. They are integrity infrastructure. They make it harder to do sloppy research, even accidentally.

Part 2: Architecture

The strategy document as constitution

`docs/strategy.md` is the single source of truth for research methodology. Every agent, skill, and review process references it. It specifies the four codebooks (C1–C4), the five-stage validation pipeline (S0–S3), success criteria, and implementation sequencing.

The strategy document lives in a **three-tier documentation system** that enforces the separation between human specification and machine context:

- **Tier 1 (auto-update):** `CLAUDE.md` files. These are execution context for Claude Code. Claude updates them directly when implementation changes make them stale. They track current status, file inventories, and target definitions.
- **Tier 2 (never auto-edit):** `docs/strategy.md`, `docs/proposal.qmd`, `docs/two_pager.qmd`. These represent deliberate research design decisions by the human. Claude never edits them directly. Instead, bottom-up discoveries are logged as entries in `docs/deltas.md`, where they accumulate until the human reconciles them via `/strategy-sync`.
- **Tier 3 (ignore):** Stable reference documents, generated artifacts, and infrastructure docs that rarely need updates.

Why this matters: The human owns the research design. The AI owns the execution context. Changes flow upward through the delta log, never through autonomous edits to the strategy.

The agent system

The project uses 10 specialized Claude Code agents in `.claude/agents/`, organized by function and deliberately constrained:

Read-only agents (cannot modify files):

Agent	Model	Role
`code-reviewer`	Haiku	R code quality check
`fiscal-policy-specialist`	Sonnet	Romer & Romer domain expert
`llm-eval-specialist`	Sonnet	Halterman & Keith evaluation expert
`strategy-reviewer`	Sonnet	Verify implementation matches strategy.md
`notebook-reviewer`	Sonnet	Verify notebooks evaluate what they should

Write-capable agents (can create and modify files):

Agent	Model	Role
`codebook-developer`	Sonnet	Draft YAML codebooks, iterate on definitions
`r-coder`	Sonnet	Write R functions for the pipeline
`pipeline-manager`	Sonnet	Manage targets pipeline definitions
`doc-writer`	Sonnet	Write Quarto documents
`document-extractor`	Haiku	PDF text extraction

Key design patterns:

- **Capability restriction matches role.** A reviewer that can also edit files is a developer pretending to review. Read-only agents are explicitly limited to `Read, Grep, Glob`.
- **Model tiering.** Haiku for cheap, narrow tasks (code review, PDF extraction). Sonnet for complex tasks requiring deeper reasoning.
- **Consultation pattern.** The `strategy-reviewer` consults `fiscal-policy-specialist` and `llm-eval-specialist` rather than trying to be an expert in everything. Specialists advise; the reviewer synthesizes.

The skill system

Skills are procedures codified as prompts, invoked via slash commands. They are not about giving Claude new capabilities. They enforce a repeatable workflow and prevent the AI from skipping steps.

Skill	Trigger	What it enforces
`/pre-flight`	Before `tar_make()`	7-check validation: git clean, target exists, dependencies current, codebook valid, API key set, cost estimate
`/review-iteration`	After pipeline run	Structured read-only diagnosis. Ends with "Do NOT propose code changes."
`/log-iteration`	After discussing results	Auto-gathers metadata (version, git hash, date), asks human for interpretation and decision, appends YAML entry
`/doc-sync`	After significant changes	Updates Tier 1 docs, logs deltas for Tier 2 docs. Nudges at 3+ unresolved deltas

Skill	Trigger	What it enforces
`/strategy-sync`	At stage gates or 3+ deltas	Adversarial reconciliation: Claude challenges, human justifies, rationale recorded
`/progress-report`	For external communication	Generates dated .qmd with auto-gathered state and terminology translation

The pipeline

The project uses `{targets}` for reproducible data processing. Two rules are absolute:

1. **All data generation goes through the pipeline.** No standalone scripts that create data files. No `saveRDS()` or `write_csv()` outside targets. This ensures reproducibility, caching, and lineage.
2. **No autonomous API calls.** Claude never runs `tar_make()` on API-calling targets without explicit human approval. Read-only operations (`tar_read()`, `tar_outdated()`, `tar_visnetwork()`) are always safe.

The pipeline tracks codebook YAML files as `format = "file"` targets, so editing a codebook automatically invalidates all downstream results.

Part 3: The Iteration Cycle

The daily workflow

The core loop for codebook development:

```
1. Edit codebook YAML (one component at a time)
2. /pre-flight          → verify everything is ready
3. tar_make(<target>)   → human authorizes the run
4. /review-iteration    → structured diagnosis (read-only)
5. Discuss results      → human interprets, decides next step
6. /log-iteration       → record what happened and why
7. Repeat or advance to next stage
```

Each step has a clear owner. Steps 1, 3, 5, and 7 are human decisions. Steps 2, 4, and 6 are Claude-assisted procedures that enforce rigor without taking control.

The documentation lifecycle

Implementation discoveries feed back into the research design through a structured cycle:

```
strategy.md (human intent)
    ↓ implementation work
docs/deltas.md (bottom-up discoveries via /doc-sync)
    ↓ accumulation trigger (3+ entries or stage gate)
/strategy-sync (adversarial reconciliation)
```

```
    ↓ disposition: incorporate / acknowledge / defer
strategy.md (updated) + audit trail (rationale recorded)
```

The strategy document stays alive rather than becoming a fossil from week one.

Exploration vs. production runs

Not all pipeline runs are equal. This project distinguishes:

- **Exploration runs** use cheap models (Qwen, Llama via OpenRouter) to test infrastructure, debug pipeline mechanics, and estimate costs. Results are not reportable.
- **Production runs** use the validated model (Claude Haiku) for formal evaluation. Results feed the iteration log and inform codebook decisions.

The iteration log explicitly marks each run's model and provider. A comment in `_targets.R` states: "Non-Anthropic providers are for cost/feasibility exploration only. Mixing providers invalidates stage comparability."

The 10 workflow conventions

These rules prevent recurring friction patterns identified during codebook development. Each exists because we learned the hard way.

1. **Plan-first mode.** When asked to diagnose or investigate, present findings and wait. Do not implement changes unless explicitly told to. *Why: prevents Claude from "fixing" things before the human understands the problem.*
2. **Root cause first.** Identify the root cause before proposing a fix. Do not patch symptoms. *Why: iteration 3 taught us that a test failure was caused by bad codebook examples, not a test implementation bug. Fixing the test would have hidden the real problem.*
3. **Model ID validation.** Verify model IDs against known valid values before writing them. *Why: stale model IDs cause silent pipeline failures that waste debugging time.*
4. **Prefer existing files.** Search with Glob/Grep before creating new files. *Why: duplicate implementations create maintenance burden and inconsistency.*
5. **No autonomous API calls.** Never run `tar_make()` on API-calling targets without explicit human approval. *Why: API calls cost money and change pipeline state. The human must authorize both.*
6. **Commit before pipeline runs.** Ensure no uncommitted changes to codebook YAML or R functions before running. *Why: the iteration log stores git hashes. Uncommitted changes break reproducibility.*
7. **One change at a time.** Change one codebook component per iteration. *Why: makes the iteration log interpretable and supports ablation-style reasoning. If you change three things and recall improves, you do not know which change helped.*

8. **Pipeline data validation.** After `tar_make()` completes, verify result shape with `tar_read(<target>) |> str()`. *Why: catches silent data issues before they propagate downstream.*
9. **Quarto render safety.** Render specific files, never the full project. *Why: prevents accidentally re-rendering expensive notebooks or breaking unrelated documents.*
10. **Strategy reconciliation.** After stage gate crossings or when 3+ unresolved deltas accumulate, run `/strategy-sync`. *Why: implementation drift is inevitable. This forces periodic reconciliation where Claude challenges your reasoning and you justify decisions on the record.*

Multi-agent workflow patterns

Three reusable patterns for parallel agent coordination:

- **Pattern 1: Codebook review.** Two specialist agents (`fiscal-policy-specialist` + `llm-eval-specialist`) review a proposed codebook change in parallel. The main session synthesizes their feedback for the user.
- **Pattern 2: Pre-pipeline validation.** `pipeline-manager` + `strategy-reviewer` verify target readiness in parallel before the user runs `tar_make()`.
- **Pattern 3: Post-stage documentation.** `doc-writer` + `notebook-reviewer` update and verify a notebook in parallel after a stage gate is crossed.

Part 4: Reference

Key file locations

Path	Purpose
`docs/strategy.md`	Authoritative methodology (Tier 2)
`docs/deltas.md`	Bottom-up implementation discoveries
`CLAUDE.md`	Project instructions for Claude Code (Tier 1)
`_targets.R`	Pipeline definition
`prompts/c1_measure_id.yml`	C1 codebook (active)
`prompts/iterations/c1.yml`	C1 iteration log
`.claude/agents/`	Agent configurations
`.claude/skills/`	Skill definitions
`R/`	Pipeline functions
`notebooks/`	Analysis and verification notebooks
`reports/`	Dated progress reports

Common commands

```
# Pipeline operations (safe, read-only)
tar_read(target_name)        # Read a target's output
tar_outdated()               # Check what needs updating
tar_visnetwork()             # Visualize dependency graph

# Pipeline execution (requires human authorization)
tar_make(target_name)        # Run a specific target

# Quarto (render specific files only)
quarto render notebooks/c1_measure_id.qmd
```

Skill quick reference

Command	When to use
`/pre-flight`	Before any `tar_make()` on API-calling targets
`/review-iteration`	After a pipeline stage completes, before deciding what to change
`/log-iteration`	After discussing results, to record the iteration
`/doc-sync`	After significant implementation work
`/strategy-sync`	At stage gates or when 3+ deltas accumulate
`/progress-report`	When preparing updates for external stakeholders

The iteration log schema

Each entry in `prompts/iterations/<codebook>.yml` records:

```
- iteration: 11
  codebook_version: "0.3.0"
  date: "2026-03-03"
  git_commit: "62495fb"
  model: "claude-haiku-4-5-20251001"
  provider: "anthropic"
  stage: "s2"
  changes: >
    What changed in the codebook since last iteration.
  results:
    overall_pass: false
    metrics:
      - metric: "combined_recall"
        value: 0.8113
        target: 0.9000
```

```
      pass: false
  interpretation: >
    What these results mean. Written by the human.
  decision: >
    What to do next. Decided by the human.
```

Every past codebook version is recoverable: `git show <hash>:prompts/c1_measure_id.yml`.